

# 深度学习图像超分辨率实验手册：使用 PyTorch 实现 SRCNN与EDSR网络

## 1. 实验概述

本实验将带领同学们完成一个完整的图像超分辨率实验。实验目标是使用 PyTorch 从零实现基础的图像超分辨率网络，并使用真实公开数据集完成训练、测试和结果分析。

本实验固定使用以下数据集：

- 训练集：DIV2K
- 测试/验证集：Set5、Set14

实验完成后，同学们应该能够理解一个深度学习图像复原项目的基本流程：

准备数据

- > 编写 Dataset
- > 定义网络模型
- > 编写损失函数和评价指标
- > 训练模型
- > 测试模型
- > 保存和观察结果

## 2. 实验原理

### 2.1 什么是图像超分辨率

图像超分辨率，也称 Super-Resolution，简称 SR，目标是从低分辨率图像恢复出更清晰的高分辨率图像。

假设：

- 低分辨率图像为 LR。
- 高分辨率真实图像为 HR。
- 模型输出图像为 SR。

那么图像超分辨率可以写成：

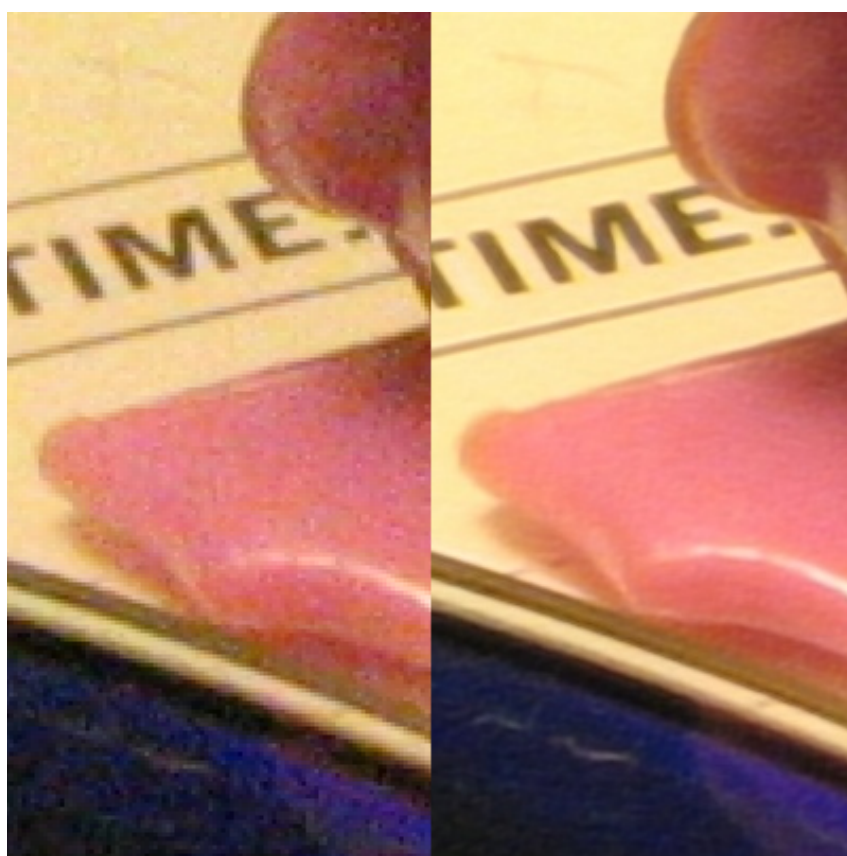
```
SR = model(LR)
```

训练时，我们希望模型输出的 SR 尽量接近真实图像 HR。

## 图像超分辨率的目标：从低分辨率恢复更清晰的高分辨率



图中可以看到，低分辨率图像的像素数量少，很多边缘和纹理信息不明显；超分辨率模型的任务就是学习如何从低分辨率输入中恢复出更接近真实 HR 图像的结果。



上图是一个真实的超分辨率效果示例：左侧是直接放大后的图像局部，右侧是超分辨率处理后的图像局部。它可以帮助同学们直观理解“放大尺寸”和“恢复细节”不是同一件事。

## 2.2 什么是插值与 Bicubic 插值

在图像处理中，插值是一种根据已有像素估计新像素的方法。它常用于图像缩放、旋转、几何变换等操作。

以图像放大为例，原图中只有有限个像素点。把一张图像放大 2 倍以后，新图像中会出现很多原图中并不存在的位置。为了给这些新位置填上合理的像素值，就需要根据周围已有像素进行估计，这个估计过程就叫插值。

常见插值方法包括：

| 方法            | 基本思想                    | 特点                  |
|---------------|-------------------------|---------------------|
| 最近邻插值         | 直接使用距离最近的原像素            | 速度快，但容易出现明显块状感      |
| 双线性插值         | 使用周围 2 x 2 个像素做加权平均     | 比最近邻平滑，但边缘容易变软      |
| Bicubic 双三次插值 | 使用周围 4 x 4 个像素做更复杂的加权估计 | 结果通常更平滑，是图像缩放中的常用方法 |

Bicubic 插值可以简单理解为：对于放大后图像中的一个新像素点，算法会找到它在原图中对应的位置，然后取这个位置附近的 16 个像素，也就是横向 4 个、纵向 4 个像素，根据距离远近分配不同权重，最后计算出新像素值。

从直观上看：

新像素值 = 周围 16 个原图像素的加权组合

距离目标位置越近的像素通常影响越大，距离越远的像素影响越小。相比最近邻和双线性插值，Bicubic 的放大结果通常更自然、更平滑，因此在传统图像缩放和超分辨率实验中经常被用作基础方法。

但是，Bicubic 插值本身不是深度学习方法，也不会从数据中学习图像纹理规律。它只能根据固定数学规则估计像素值，所以虽然能把图像放大到目标尺寸，但往往无法真正恢复出清晰的高频细节。SRCNN 的作用正是在 Bicubic 放大结果的基础上，进一步学习如何修复边缘和纹理。

## 2.3 本实验中的输入和输出

为了让实验流程更简单，本实验采用 SRCNN 中常见的一种做法：

- 1. 先把 HR 图像缩小，模拟低分辨率图像 LR。
- 2. 再用 Bicubic 插值把 LR 放大回 HR 的尺寸。
- 3. 把放大后的图像送入 SRCNN。
- 4. SRCNN 输出一张修复后的图像。
- 5. 使用真实 HR 图像作为监督信号训练网络。

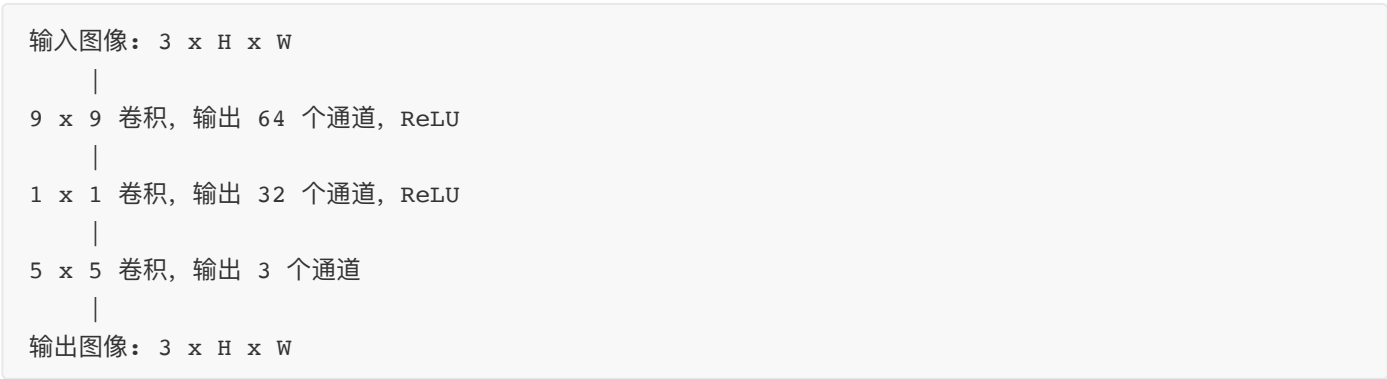


也就是说，模型真正学习的是：

Bicubic 放大后的模糊图像 -> 更接近 HR 的清晰图像

## 2.4 SRCNN 网络结构

本实验实现 RGB 三通道版本的 SRCNN。网络只有三层卷积：



这张图对应 `model.py` 中的三层卷积代码。阅读模型代码时，可以把每一层卷积和图中的一个模块对应起来看。

三层卷积可以这样理解：

- 1. 第一层从图像中提取边缘、纹理等局部特征。
- 2. 第二层对这些特征进行组合和非线性变换。
- 3. 第三层把特征重新转换成 RGB 图像。

## 2.5 损失函数

本实验使用均方误差损失 MSE：

```
MSE = mean((SR - HR)^2)
```

其中：

- SR 是模型输出。
- HR 是真实高分辨率图像。

MSE 越小，说明模型输出的像素值越接近真实图像。

## 2.6 评价指标 PSNR

本实验使用 PSNR 评价图像恢复质量。

当图像像素范围是 0 到 1 时：

```
PSNR = 10 * log10(1 / MSE)
```

PSNR 越高，通常表示模型输出越接近真实 HR 图像。

说明：原始 SRCNN 论文和很多超分辨率项目常在 YCbCr 的 Y 通道上计算 PSNR，并且会裁剪图像边界。本实验为了让代码更容易理解，直接在 RGB 图像上计算 PSNR。

## 3. 环境搭建

本节参考常见 SRCNN PyTorch GitHub 项目的说明方式：先创建 Python 环境，再安装 PyTorch、Pillow、NumPy、tqdm 等基础依赖，最后运行训练和测试脚本。

### 3.1 推荐环境

建议环境如下：

| 项目     | 建议                             |
|--------|--------------------------------|
| Python | 3.10 或 3.11                    |
| 深度学习框架 | PyTorch 2.x                    |
| GPU    | 推荐 NVIDIA GPU，没有 GPU 也可以使用 CPU |
| 系统     | Windows、macOS、Linux 均可         |

如果使用 Windows，建议使用 Anaconda Prompt、PowerShell、Git Bash 或 WSL 执行命令。

### 3.2 创建虚拟环境（使用云服务器中的base环境即可）

使用 Conda：

```
conda create -n srcnn_lab python=3.10 -y
conda activate srcnn_lab
```

### 3.3 安装 PyTorch（云服务器中已有，可跳过）

如果只使用 CPU，可以执行：

```
pip install torch torchvision
```

如果使用 NVIDIA GPU，请打开 PyTorch 官方安装页面，根据自己的 CUDA 版本生成安装命令：

```
https://pytorch.org/get-started/locally/
```

例如，官网页面会根据系统、包管理器、CUDA 版本生成类似下面的命令：

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
```

注意：CUDA 版本可能随时间更新，GPU 用户应优先使用 PyTorch 官网生成的最新命令。

## 3.4 安装其他依赖

本实验代码只需要几个常用库：

```
pip install pillow numpy tqdm
```

依赖说明：

| 库                        | 作用                              |
|--------------------------|---------------------------------|
| <code>torch</code>       | 搭建和训练神经网络                       |
| <code>torchvision</code> | PyTorch 视觉相关基础库，本实验主要作为常用配套依赖安装 |
| <code>pillow</code>      | 读取、缩放和保存图像                      |
| <code>numpy</code>       | 图像数组和 Tensor 转换                 |
| <code>tqdm</code>        | 显示训练进度条                         |

## 3.5 检查环境

执行：

```
python -c "import torch; import PIL; import numpy; print('torch:', torch.__version__);  
print('cuda:', torch.cuda.is_available())"
```

如果能正常输出 PyTorch 版本，说明环境安装成功。

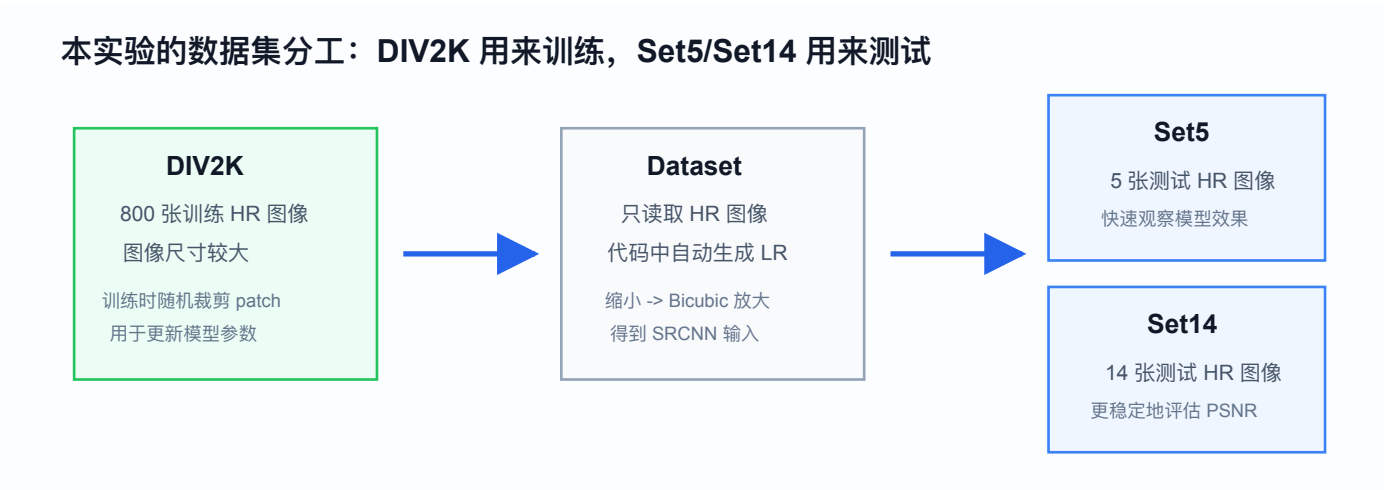
如果 `cuda` 输出为 `False`，表示当前 PyTorch 没有检测到可用 GPU。没有 GPU 时仍然可以完成实验，只是训练速度会慢一些。

# 4. 数据准备

本实验使用 DIV2K 进行训练，使用 Set5 和 Set14 进行测试验证。

## 4.1 数据集说明

| 数据集   | 用途    | 说明                      |
|-------|-------|-------------------------|
| DIV2K | 训练    | 高质量 2K 图像数据集，常用于超分辨率训练  |
| Set5  | 测试/验证 | 只有 5 张图，常用于超分辨率快速测试     |
| Set14 | 测试/验证 | 14 张图，比 Set5 略大，也是常用测试集 |



这张图说了三个数据集在实验中的角色：DIV2K 用于训练模型参数，Set5 和 Set14 用于测试模型效果，不参与参数更新。

本实验只需要这些数据集中的 HR 图像。LR 图像会在代码中自动生成。

## 4.2 创建数据目录

创建项目根目录

```
mkdir -p srcnn
cd srcnn
```

然后进入目录执行：

```
mkdir -p data/raw
mkdir -p data/DIV2K
mkdir -p data/benchmark/Set5/HR
mkdir -p data/benchmark/Set14/HR
mkdir -p outputs/checkpoints
mkdir -p outputs/results
mkdir -p src
```

目录含义：

- data/DIV2K/ 用来保存 DIV2K 训练图像。
- data/benchmark/Set5/HR/ 用来保存 Set5 的 HR 图像。
- data/benchmark/Set14/HR/ 用来保存 Set14 的 HR 图像。
- outputs/checkpoints/ 用来保存训练好的模型。

- `outputs/results/` 用来保存测试输出图片。

## 4.3 下载 DIV2K 训练集

使用 Hugging Face 镜像下载，通常比 DIV2K 官网更容易连接。

```
cd data
git clone https://github.com/Benjamin-Wegener/DIV2K
rm DIV2K/DIV2K_train_HR/0001.png
cd ..
```

下载后应该得到：

```
data/DIV2K/DIV2K_train_HR/
```

该目录中应该有 800 张训练图像，文件名为：

```
0001.jpeg, 0002.jpeg, ..., 0800.jpeg
```

可以用下面的命令检查数量：

```
find data/DIV2K/DIV2K_train_HR -type f -name "*.jpeg" | wc -l
```

也可以检查前几张图片的文件名：

```
find data/DIV2K/DIV2K_train_HR -type f -name "*.jpeg" | sort | head
```

## 4.4 下载 Set5 和 Set14

Set5、Set14 可以从 SelfExSR 的 GitHub 仓库中获取。该仓库的 `data/` 目录中包含 Set5、Set14、BSD100、Urban100 等常见超分辨率测试数据集。

下载仓库：

```
cd data/raw
git clone https://github.com/jbhuang0604/SelfExSR
cd ../..
```

下载后，Set5 和 Set14 的 x2 数据位于：

```
data/raw/SelfExSR/data/Set5/image_SRF_2/
data/raw/SelfExSR/data/Set14/image_SRF_2/
```

其中：

- 文件名中包含 `_HR` 的图片是高分辨率图像。
- 文件名中包含 `_LR` 的图片是低分辨率图像。



本实验只需要 HR 图像，因为 LR 图像会在代码中自动生成。

复制 Set5 和 Set14 的 HR 图像：

```
cp data/raw/SelfExSR/data/Set5/image_SRF_2/*_HR.png data/benchmark/Set5/HR/
cp data/raw/SelfExSR/data/Set14/image_SRF_2/*_HR.png data/benchmark/Set14/HR/
```

检查文件数量：

```
find data/benchmark/Set5/HR -type f -name "*_HR.png" | wc -l
find data/benchmark/Set14/HR -type f -name "*_HR.png" | wc -l
```

正常情况下：

- Set5 应该有 5 张 HR 图像。
- Set14 应该有 14 张 HR 图像。

如果想确认原始文件结构，可以执行：

```
find data/raw/SelfExSR/data/Set5/image_SRF_2 -type f | head
find data/raw/SelfExSR/data/Set14/image_SRF_2 -type f | head
```

## 4.5 数据准备后的目录

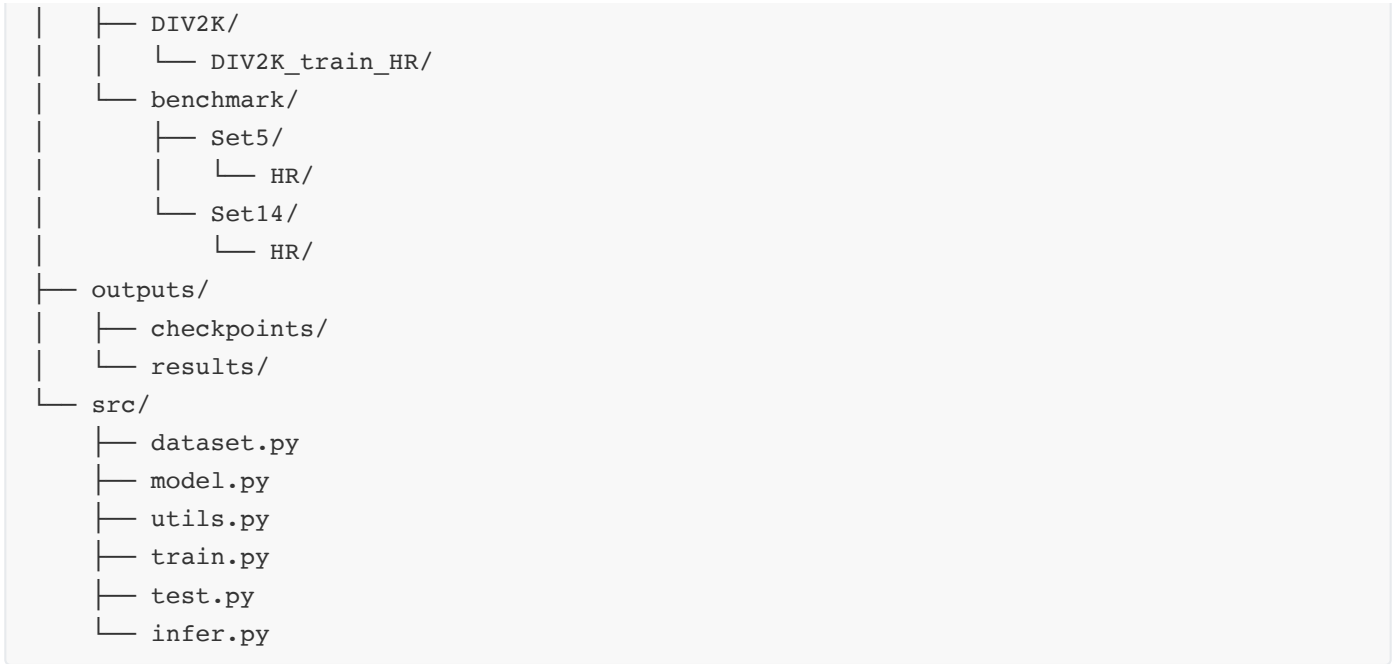
完成数据准备后，项目中的数据部分应大致如下：

```
data/
├── DIV2K/
│   └── DIV2K_train_HR/
│       ├── 0001.jpeg
│       ├── 0002.jpeg
│       └── ...
└── benchmark/
    ├── Set5/
    │   └── HR/
    │       ├── img_001_SRF_2_HR.png
    │       └── ...
    └── Set14/
        └── HR/
            ├── img_001_SRF_2_HR.png
            └── ...
```

## 5. 项目结构

建议创建如下项目结构：

```
srcnn_sr_lab/
├── data/
```



本实验不额外编写依赖清单文件，所有安装命令已经在环境搭建部分给出。

## 6. 完整代码

本实验代码分成 6 个文件：

| 文件                      | 作用                              |
|-------------------------|---------------------------------|
| <code>utils.py</code>   | 图像和 Tensor 转换、PSNR 计算、拼接对比图     |
| <code>dataset.py</code> | 读取 HR 图像，并在代码中生成 LR 输入          |
| <code>model.py</code>   | 定义 SRCNN 网络                     |
| <code>train.py</code>   | 使用 DIV2K 训练模型，并在 Set5/Set14 上验证 |
| <code>test.py</code>    | 加载模型，在 Set5/Set14 上测试并保存结果      |
| <code>infer.py</code>   | 对单张图片进行超分辨率推理                   |

### 6.1 工具函数 utils.py

文件路径：

```
src/Utils.py
```

完整代码：

```
# math 提供数学函数，这里主要用 log10 计算 PSNR。
import math
```

```

# pathlib.Path 是 Python 中更现代的路径处理工具。
# 相比直接使用字符串路径, Path 可以更方便地拼接目录、创建文件夹、遍历文件。
from pathlib import Path

# numpy 用于处理图像数组。
# PIL 读入的图像会先转成 NumPy 数组, 再转成 PyTorch Tensor。
import numpy as np

# torch 是 PyTorch 的核心库, 本实验的神经网络、Tensor 都基于它。
import torch

# Image 用于读取、保存、缩放图像。
# ImageDraw 用于在对比图上写文字标签。
from PIL import Image, ImageDraw

def list_image_files(folder):
    """
    找到一个文件夹下面所有常见格式的图像文件。

    参数:
        folder: 图像文件夹路径。

    返回:
        一个按文件名排序后的 Path 列表。

    为什么要写这个函数:
        训练和测试时, 我们只希望 Dataset 关心图像路径列表,
        不希望在 Dataset 里面写很多重复的文件搜索代码。
    """
    # 把传入的 folder 转换为 Path 对象。
    # 这样无论传入的是字符串 "data/xxx", 还是 Path("data/xxx"),
    # 后面的代码都可以用统一方式处理。
    folder = Path(folder)

    # 这里列出允许读取的图像后缀。
    # lower() 后再比较, 所以 .PNG、.JPG 这类大写后缀也能被识别。
    image_exts = {".png", ".jpg", ".jpeg", ".bmp"}

    # files 用来保存找到的所有图像路径。
    files = []

    # rglob("*") 表示递归遍历 folder 下面的所有文件和子文件夹。
    # 如果数据集目录下面还有子目录, 这种写法也能找到其中的图像。
    for path in folder.rglob("*"):
        # path.suffix 表示文件后缀, 例如 ".png"。
        # lower() 表示转成小写, 避免因为 ".PNG" 这类大写后缀漏掉文件。
        if path.suffix.lower() in image_exts:
            files.append(path)

    # sorted(files) 会按路径名排序。

```

# 这样每次运行时图像顺序稳定，便于复现实验结果和排查问题。

```
return sorted(files)
```

```
def image_to_tensor(image):
```

```
    """
```

把 PIL 图像转换为 PyTorch Tensor。

PIL 图像的形状可以理解为：

H x W x C

PyTorch 卷积网络要求的输入形状是：

C x H x W

所以这里需要做两件事：

1. 像素值从 0~255 转成 0~1。

2. 维度顺序从 HWC 改成 CHW。

```
    """
```

# convert("RGB") 保证图像一定是三通道。

# 有些 png 可能带透明通道 RGBA，有些灰度图可能只有一个通道，

# 统一成 RGB 后，网络输入通道数就固定为 3。

```
image = image.convert("RGB")
```

# np.array 得到的数组形状是 H x W x C，数值范围是 0 到 255。

```
array = np.array(image).astype(np.float32)
```

# 神经网络训练通常使用 0 到 1 的浮点数，而不是 0 到 255 的整数。

# 除以 255.0 后，黑色约为 0，白色约为 1。

```
array = array / 255.0
```

# transpose(2, 0, 1) 表示把 HWC 改成 CHW。

# 原来的三个维度编号分别是：

# 0: H, 高度

# 1: W, 宽度

# 2: C, 通道

# 变成 (2, 0, 1) 后就是 C x H x W。

```
array = array.transpose(2, 0, 1)
```

# torch.from\_numpy 把 NumPy 数组变成 PyTorch Tensor。

# 这里得到的 Tensor 形状是 3 x H x W。

```
return torch.from_numpy(array)
```

```
def tensor_to_image(tensor):
```

```
    """
```

把 PyTorch Tensor 转换回 PIL 图像。

输入 Tensor 的形状：

C x H x W

输出 PIL 图像的形状：

H x W x C

这个函数主要用于保存模型输出图片。

```
"""
# detach 表示从计算图中分离。
# 保存图片时不需要梯度信息。
# cpu 表示移动到 CPU。
# PIL 和 NumPy 不能直接处理 GPU Tensor。
# clamp 表示限制像素范围。
# 模型输出有时会略小于 0 或略大于 1，保存前需要裁剪回合法范围。
tensor = tensor.detach().cpu().clamp(0.0, 1.0)

# 把 CHW 改回 HWC。
# PIL 保存图片时需要 H x W x C 的数组。
array = tensor.numpy().transpose(1, 2, 0)

# 把 0~1 浮点数改回 0~255 整数。
# round() 是四舍五入，astype(np.uint8) 转成图像常用的 8 位整数。
array = (array * 255.0).round().astype(np.uint8)

# Image.fromarray 把 NumPy 数组重新变成 PIL 图像。
return Image.fromarray(array)
```

```
def calc_psnr(sr, hr):
```

```
"""
```

计算 SR 图像和 HR 图像之间的 PSNR。

参数：

sr：模型输出图像 Tensor，像素范围建议是 0 到 1。

hr：真实高分辨率图像 Tensor，像素范围是 0 到 1。

返回：

一个浮点数，单位是 dB。

```
"""
```

```
# 评价指标只需要数值，不需要梯度，所以先 detach。
# clamp 可以避免少量越界像素影响 MSE 和 PSNR 计算。
sr = sr.detach().clamp(0.0, 1.0)
hr = hr.detach().clamp(0.0, 1.0)

# MSE 表示均方误差。
# (sr - hr) 是每个像素的误差，平方后再求平均。
# .item() 把只有一个数的 Tensor 转成 Python 浮点数，方便后面计算 log10。
mse = torch.mean((sr - hr) ** 2).item()

# 如果两张图完全一样，MSE 为 0。
# 此时 1 / mse 没法计算，理论上 PSNR 是无穷大。
if mse == 0:
    return float("inf")

# 当图像像素范围是 0~1 时，最大像素值 MAX_I = 1。
# PSNR = 10 * log10(MAX_I^2 / MSE)，所以这里写成 10 * log10(1 / MSE)。
return 10.0 * math.log10(1.0 / mse)
```

```
def save_compare_image(lr_image, sr_image, hr_image, save_path):
    """
    保存一张横向对比图。

    对比图内容：
        Bicubic 输入 | SRCNN 输出 | HR 真值

    参数中的 lr_image、sr_image、hr_image 都是 PIL 图像。
    """
    # 这里以 HR 图像的宽高作为对比图中每一列的宽高。
    width, height = hr_image.size

    # 给顶部文字标签预留一小段高度。
    label_height = 28

    # 创建一张白色画布，宽度是三张图并排，高度是图像高度加上标签高度。
    canvas = Image.new("RGB", (width * 3, height + label_height), "white")

    # paste 的第二个参数是粘贴位置左上角坐标。
    # 第一张图放在第 0 列，第二张图放在第 1 列，第三张图放在第 2 列。
    canvas.paste(lr_image, (0, label_height))
    canvas.paste(sr_image, (width, label_height))
    canvas.paste(hr_image, (width * 2, label_height))

    # 在画布顶部写上每一列的名称。
    draw = ImageDraw.Draw(canvas)
    draw.text((10, 8), "Bicubic", fill=(0, 0, 0))
    draw.text((width + 10, 8), "SRCNN", fill=(0, 0, 0))
    draw.text((width * 2 + 10, 8), "HR", fill=(0, 0, 0))

    # 确保输出目录存在。
    # parents=True 表示如果上级目录不存在也一起创建。
    # exist_ok=True 表示目录已经存在时不报错。
    save_path = Path(save_path)
    save_path.parent.mkdir(parents=True, exist_ok=True)

    # 保存最终对比图。
    canvas.save(save_path)
```

重点理解：

- `image_to_tensor` 和 `tensor_to_image` 是图像任务中非常常见的两个函数。
- 神经网络中常用 `C x H x W`，普通图像文件中更常见的是 `H x W x C`。
- PSNR 是用 MSE 推导出来的，MSE 越小，PSNR 越大。

## 6.2 数据集 dataset.py

文件路径：

```
src/dataset.py
```

完整代码：

```
import random

# PIL 的 Image 用于打开图像、裁剪图像、缩放图像。
from PIL import Image

# Dataset 是 PyTorch 提供的数据集基类。
# 自定义 Dataset 时，通常需要继承它，并实现 __len__ 和 __getitem__。
from torch.utils.data import Dataset

# image_to_tensor 把 PIL 图像转成 Tensor。
# list_image_files 用于找到文件夹中的所有图像文件。
from utils import image_to_tensor, list_image_files


class SRDataset(Dataset):
    """
    图像超分辨率 Dataset。

    这个 Dataset 只读取 HR 图像。
    LR 图像不是提前保存好的，而是在 __getitem__ 中动态生成。
    为了减少训练时反复读取硬盘的时间，初始化时会把所有 HR 图像
    一次性读取到内存中。

    这样做的好处：
        1. 数据准备更简单，只需要准备 HR 图像。
        2. 训练时不需要每次都从硬盘打开图片，速度更快。
        3. 训练时每次随机裁剪 patch，可以得到更多训练样本。
        4. 初学者更容易看懂 LR 和 HR 的对应关系。

    注意：
        DIV2K 图像较大，一次性加载到内存会占用较多 RAM。
        如果电脑内存较小，可以改回在 __getitem__ 中按需读取图像。
    """

    def __init__(self, hr_dir, scale=2, patch_size=96, train=True,
                 samples_per_epoch=2000):
        """
        参数：
            hr_dir: HR 图像所在文件夹。
            scale: 放大倍数，例如 scale=2 表示 2 倍超分辨率。
            patch_size: 训练时从 HR 图像中裁剪的小图块大小。
            train: True 表示训练模式，False 表示测试模式。
            samples_per_epoch: 训练模式下每个 epoch 取多少个随机 patch。
        """
        # 找到 HR 文件夹中的所有图像路径。
        # 这里保存的是路径列表，不是图像内容。
        self.image_files = list_image_files(hr_dir)
```

```

# 把传入的参数保存成成员变量。
# 这样类中的其他函数，例如 random_crop 和 make_lr_input,
# 就可以通过 self.scale、self.patch_size 使用这些设置。
self.scale = scale
self.patch_size = patch_size
self.train = train
self.samples_per_epoch = samples_per_epoch

# 如果路径写错，或者数据集没有下载成功，这里会直接报错。
# 这样可以尽早发现问题，而不是等到训练循环中才出错。
if len(self.image_files) == 0:
    raise RuntimeError(f"No image files found in {hr_dir}")

# patch_size 必须能被 scale 整除。
# 例如 scale=4 时，patch_size=96 可以，因为 96/4=24。
# 如果不能整除，先下采样再上采样后可能出现尺寸对不齐。
if patch_size % scale != 0:
    raise ValueError("patch_size must be divisible by scale")

# self.images 用来保存已经读入内存的图像。
# 每个元素是一个字典，里面包含图像名和 PIL 图像对象。
self.images = []
print(f>Loading {len(self.image_files)} HR images into memory...")

for path in self.image_files:
    # Image.open(path) 打开图像文件。
    # convert("RGB") 统一成三通道，避免灰度图或带透明通道的图像造成输入通道不一致。
    image = Image.open(path).convert("RGB")

    # 预先把图像裁剪到能被 scale 整除的尺寸。
    # 这样后面生成 LR 和 LR_up 时，尺寸会比较规整。
    image = self.crop_to_multiple(image)

    # path.stem 是不带后缀的文件名。
    # 例如 0801.png 的 stem 是 0801，后面保存结果图时会用到。
    self.images.append(
        {
            "name": path.stem,
            "image": image,
        }
    )

print("Finished loading images.")

def __len__(self):
    """
    返回 Dataset 的长度。

    测试模式：
        有多少张图，就返回多少。

```



训练模式：

DIV2K 只有 800 张 HR 图像，但每张图很大。

我们每个 epoch 随机裁剪 samples\_per\_epoch 个 patch，

因此训练长度设置为 samples\_per\_epoch。

"""

# 训练模式下，Dataset 的长度不等于图片张数。

# 因为每张大图可以随机裁剪出很多 patch。

if self.train:

return self.samples\_per\_epoch

# 测试模式下不随机裁剪，一张测试图就是一个样本。

return len(self.images)

def crop\_to\_multiple(self, image):

"""

把图像裁剪到宽和高都能被 scale 整除。

为什么需要这一步：

如果宽高不能被 scale 整除，下采样再上采样时尺寸可能对不上。

"""

# PIL 中 image.size 返回的是 (宽度, 高度), 即 (W, H)。

width, height = image.size

# width % self.scale 表示宽度除以 scale 后的余数。

# 减掉这个余数后, 新的 width 就一定被 scale 整除。

width = width - width % self.scale

height = height - height % self.scale

# crop((left, top, right, bottom)) 表示从左上角裁剪到新的宽高。

return image.crop((0, 0, width, height))

def random\_crop(self, image):

"""

从 HR 图像中随机裁剪一个 patch。

训练时不直接使用整张 DIV2K 图像，原因是：

1. 原图很大，直接训练显存消耗高。
2. 随机裁剪可以让模型看到更多不同位置的局部纹理。

"""

width, height = image.size

# 如果原图比 patch\_size 还小，无法随机裁剪出指定大小的 patch。

# 为了让代码保持简单，这里直接把小图缩放到 patch\_size。

if width < self.patch\_size or height < self.patch\_size:

image = image.resize((self.patch\_size, self.patch\_size), Image.BICUBIC)

return image

# 随机选择裁剪区域左上角坐标。

# left 的最大值是 width - patch\_size,

# 这样 right = left + patch\_size 不会超过图像右边界。

left = random.randint(0, width - self.patch\_size)

top = random.randint(0, height - self.patch\_size)

```

# 根据左上角坐标计算右下角坐标。
right = left + self.patch_size
bottom = top + self.patch_size

# 返回裁剪后的 HR patch。
return image.crop((left, top, right, bottom))

def make_lr_input(self, hr):
    """
    根据 HR 图像生成模型输入。

    步骤：
        1. 把 HR 缩小 scale 倍，得到低分辨率图像。
        2. 再用 Bicubic 插值放大回原尺寸。

    注意：
        SRCNN 的输入和输出尺寸相同。
        这里返回的是 Bicubic 放大后的图像，而不是小尺寸 LR 图像。
    """
    # hr.size 返回当前 HR 图像的宽和高。
    width, height = hr.size

    # 先计算低分辨率图像的尺寸。
    # 例如 scale=4, 原图 128 x 128 会变成 32 x 32。
    lr_width = width // self.scale
    lr_height = height // self.scale

    # 第一次 resize: HR -> LR。
    # 这一步模拟真实低分辨率图像，也就是故意降低图像质量。
    lr = hr.resize((lr_width, lr_height), Image.BICUBIC)

    # 第二次 resize: LR -> 原尺寸。
    # SRCNN 接收已经放大到目标尺寸的图像。
    # 第 9 节的 PixelShuffle 版 EDSR 会使用单独的 dataset_edsr.py,
    # 它直接返回小尺寸 LR 图像，并由 EDSR 网络内部完成上采样。
    lr_up = lr.resize((width, height), Image.BICUBIC)

    return lr_up

def __getitem__(self, index):
    """
    取出一个训练或测试样本。

    返回字典：
        lr:    Bicubic 放大后的输入图像 Tensor
        hr:    真实高分辨率图像 Tensor
        name:  图像文件名，用于保存结果
    """
    # 训练模式下 __len__ 返回 samples_per_epoch, 可能大于真实图片数。
    # 因此使用 index % len(self.images) 让索引循环回到图片列表范围内。
    item = self.images[index % len(self.images)]

```

```

# 从内存中取出的 PIL 图像是原始图像对象。
# 这里使用 copy() 得到一个副本，避免后续裁剪操作影响内存中的原图。
hr = item["image"].copy()

# 训练时使用随机 patch。
# 测试时不进入这个分支，直接使用完整图像。
if self.train:
    hr = self.random_crop(hr)

# 根据 HR 生成 Bicubic 放大后的输入图像。
lr_up = self.make_lr_input(hr)

# 返回字典形式的数据。
# DataLoader 会把多个样本的 lr 自动拼成 B x 3 x H x W 的 batch。
return {
    "lr": image_to_tensor(lr_up),
    "hr": image_to_tensor(hr),
    "name": item["name"],
}

```

重点理解：

- Dataset 的核心是 `__len__` 和 `__getitem__`。
- `__getitem__` 每次返回一组输入和标签。
- 在前面的 SRCNN 实验中，输入是 Bicubic 放大图像，标签是 HR 图像。
- 训练时使用随机 patch，测试时使用完整图像。
- `__init__` 中会把 HR 图像一次性读入内存，这样训练时不用反复从硬盘读取图片，速度会更快。
- 如果电脑内存不足，可以把 `self.images` 这部分去掉，恢复成在 `__getitem__` 中使用 `Image.open(path)` 按需读取。

## 6.3 模型 model.py

文件路径：

```
src/model.py
```

完整代码：

```

# torch.nn 里面包含神经网络常用层，例如卷积层、激活函数、损失函数等。
import torch.nn as nn

class SRCNN(nn.Module):
    """
    SRCNN 网络。

```

输入：

`B x 3 x H x W`

输出：

`B x 3 x H x W`

其中：

`B` 是 batch size。

`3` 表示 RGB 三个通道。

`H` 和 `W` 是图像高度和宽度。

"""

```
def __init__(self):
```

```
    # super().__init__() 会初始化 nn.Module 父类中的内容。
```

```
    # 只要自己写 PyTorch 模型类，通常都需要先调用这一句。
```

```
    super().__init__()
```

```
    # nn.Sequential 可以把多个网络层按顺序串起来。
```

```
    # 前一层的输出会自动作为后一层的输入。
```

```
    self.net = nn.Sequential(
```

```
        # 第一层卷积：
```

```
        # 输入是 RGB 图像，所以 in_channels=3。
```

```
        # 输出 64 个特征通道，相当于提取 64 类局部特征。
```

```
        # kernel_size=9 表示卷积核大小是 9 x 9。
```

```
        # padding=4 可以保证输出图像尺寸不变。
```

```
        nn.Conv2d(3, 64, kernel_size=9, padding=4),
```

```
        # ReLU 是非线性激活函数。
```

```
        # inplace=True 表示尽量原地修改数据，可稍微节省显存。
```

```
        nn.ReLU(inplace=True),
```

```
        # 第二层卷积：
```

```
        # 1 x 1 卷积不会看周围像素，只在通道维度上做信息融合。
```

```
        # 它把 64 个通道压缩到 32 个通道。
```

```
        nn.Conv2d(64, 32, kernel_size=1, padding=0),
```

```
        # 再做一次非线性变换。
```

```
        nn.ReLU(inplace=True),
```

```
        # 第三层卷积：
```

```
        # 把 32 个特征通道重新映射回 RGB 三通道图像。
```

```
        # padding=2 可以保证 5 x 5 卷积后尺寸不变。
```

```
        nn.Conv2d(32, 3, kernel_size=5, padding=2),
```

```
    )
```

```
def forward(self, x):
```

```
    """
```

```
    前向传播。
```

```
    x 是输入图像 Tensor。
```

```
    self.net(x) 会依次执行三层卷积和两次 ReLU。
```

```
    """
```

```
# 调用 self.net(x) 后，输入会依次经过上面定义的所有层。
return self.net(x)
```

重点理解：

- SRCNN 的代码非常短，因为网络结构很简单。
- `padding` 的作用是保持输入和输出的空间尺寸一致。
- 最后一层后面不加 ReLU，因为输出图像像素值会在保存和计算指标时再限制到 0 到 1。

## 6.4 训练 train.py

文件路径：

```
src/train.py
```

完整代码：

```
# argparse 用于读取命令行参数。
# 例如 --epochs 50、--batch-size 16 都会被 argparse 解析到 args 里面。
import argparse

# Path 用于创建保存模型的文件夹。
from pathlib import Path

# torch 是 PyTorch 主库。
import torch

# torch.nn 提供损失函数等神经网络组件。
import torch.nn as nn

# torch.optim 提供优化器，例如 Adam、SGD。
import torch.optim as optim

# DataLoader 负责把 Dataset 里的单个样本组织成 batch。
from torch.utils.data import DataLoader

# tqdm 用于显示训练进度条。
from tqdm import tqdm

from dataset import SRDataset
from model import SRCNN
from utils import calc_psnr

@torch.no_grad()
def evaluate(model, data_loader, device):
    """
    在一个测试集上计算平均 PSNR。
```

@torch.no\_grad() 表示不计算梯度。

测试和验证阶段不需要反向传播，所以这样可以节省显存和时间。

```
"""
```

# model.eval() 会把模型切换到验证/测试模式。

# 本实验中的 SRCNN 没有 BatchNorm 或 Dropout，但保留这句是良好习惯。

```
model.eval()
```

# total\_psnr 用于累加每张测试图像的 PSNR。

```
total_psnr = 0.0
```

# image\_count 用于记录一共测试了多少张图像。

```
image_count = 0
```

```
for batch in data_loader:
```

```
    # batch["lr"] 的形状是 B x 3 x H x W。
```

```
    # .to(device) 表示把数据移动到 CPU 或 GPU，与模型所在设备保持一致。
```

```
    lr = batch["lr"].to(device)
```

```
    hr = batch["hr"].to(device)
```

```
    # 前向传播，得到模型输出的超分辨率图像。
```

```
    sr = model(lr)
```

```
    # 计算当前 batch 的 PSNR。
```

```
    # 这里测试时 batch_size=1，所以可以理解为一张图的 PSNR。
```

```
    psnr = calc_psnr(sr, hr)
```

```
    total_psnr += psnr
```

```
    image_count += 1
```

```
# 返回测试集上的平均 PSNR。
```

```
return total_psnr / image_count
```

```
def train(args):
```

```
    """
```

训练主函数。

主要流程：

1. 创建 Dataset 和 DataLoader。
2. 创建 SRCNN 模型。
3. 定义损失函数和优化器。
4. 循环训练多个 epoch。
5. 每个 epoch 后在 Set5 和 Set14 上计算 PSNR。
6. 保存最好的模型。

```
    """
```

```
# 如果有 CUDA 显卡，并且用户没有手动指定 --cpu，就使用 GPU。
```

```
# 否则使用 CPU。torch.device 会把设备名称封装成 PyTorch 可识别的对象。
```

```
device = torch.device("cuda" if torch.cuda.is_available() and not args.cpu else "cpu")
```

```
print(f"Using device: {device}")
```

```
# 创建训练集。
```

```
# 这里使用 DIV2K 的 HR 图像，Dataset 会在内部自动生成 LR 输入。
```

```
train_set = SRDataset(
```

```

    hr_dir=args.train_dir,
    scale=args.scale,
    patch_size=args.patch_size,
    train=True,
    samples_per_epoch=args.samples_per_epoch,
)

# 创建两个验证集。
# train=False 表示不随机裁剪，直接使用完整测试图像。
set5 = SRDataset(args.set5_dir, scale=args.scale, train=False)
set14 = SRDataset(args.set14_dir, scale=args.scale, train=False)

# DataLoader 会不断调用 train_set.__getitem__,
# 并把多个样本拼成一个 batch。
train_loader = DataLoader(
    train_set,

    # batch_size 表示每次送入模型的图像 patch 数量。
    batch_size=args.batch_size,

    # shuffle=True 表示每个 epoch 打乱训练样本顺序，有助于训练更稳定。
    shuffle=True,

    # num_workers=0 表示在主进程中读取数据。
    # 对初学实验来说最简单，也更容易在 Windows/macOS 上运行。
    num_workers=0,
)

# 测试时通常 batch_size=1。
# 因为测试图像尺寸不同，batch_size=1 可以避免尺寸不一致导致无法拼 batch。
set5_loader = DataLoader(set5, batch_size=1, shuffle=False, num_workers=0)
set14_loader = DataLoader(set14, batch_size=1, shuffle=False, num_workers=0)

# 创建 SRCNN 模型，并移动到指定设备。
model = SRCNN().to(device)

# MSELoss 会计算模型输出 SR 和真实图像 HR 之间的均方误差。
loss_fn = nn.MSELoss()

# Adam 是常用优化器，lr 是学习率。
optimizer = optim.Adam(model.parameters(), lr=args.lr)

# 创建模型保存目录。
# 训练过程中会在这里保存 last.pth 和 best.pth。
save_dir = Path(args.save_dir)
save_dir.mkdir(parents=True, exist_ok=True)

# best_score 记录目前为止最好的平均 PSNR。
# 只有新的模型超过它时，才保存为 best.pth。
best_score = 0.0

# epoch 表示完整遍历一轮训练样本。

```

```

# range(1, args.epochs + 1) 让 epoch 从 1 开始计数, 更符合日志阅读习惯。
for epoch in range(1, args.epochs + 1):
    # model.train() 会把模型切换到训练模式。
    model.train()

    # total_loss 用于累加当前 epoch 中所有 batch 的 loss。
    total_loss = 0.0

    # tqdm 包装 DataLoader 后, 会显示当前 epoch 的训练进度。
    progress = tqdm(train_loader, desc=f"Epoch {epoch}/{args.epochs}")

    for batch in progress:
        # 从 batch 字典中取出输入 lr 和标签 hr。
        # lr、hr 都要移动到和模型相同的 device 上。
        lr = batch["lr"].to(device)
        hr = batch["hr"].to(device)

        # 前向传播: 输入 lr, 得到模型预测 sr。
        sr = model(lr)

        # 计算预测图像 sr 和真实图像 hr 的差距。
        loss = loss_fn(sr, hr)

        # 清空上一次迭代留下的梯度。
        optimizer.zero_grad()

        # 反向传播, 计算每个参数的梯度。
        loss.backward()

        # 根据梯度更新模型参数。
        optimizer.step()

        # loss.item() 把 Tensor 类型的 loss 转成普通浮点数, 方便累加和打印。
        total_loss += loss.item()

        # 在进度条右侧显示当前 batch 的 loss。
        progress.set_postfix(loss=f"{loss.item():.6f}")

    # 当前 epoch 的平均 loss。
    avg_loss = total_loss / len(train_loader)

    # 每个 epoch 结束后, 在 Set5 和 Set14 上验证一次。
    set5_psnr = evaluate(model, set5_loader, device)
    set14_psnr = evaluate(model, set14_loader, device)

    # 用两个测试集 PSNR 的平均值作为判断 best.pth 的依据。
    mean_psnr = (set5_psnr + set14_psnr) / 2.0

    print(
        f"Epoch {epoch:03d} | "
        f"loss={avg_loss:.6f} | "
        f"Set5={set5_psnr:.2f} dB | "

```



```

        f"Set14={set14_psnr:.2f} dB"
    )

    # checkpoint 是一个字典，里面保存训练后的模型参数和一些实验信息。
    checkpoint = {
        # state_dict 保存模型中所有可学习参数。
        "model": model.state_dict(),
        "epoch": epoch,
        "scale": args.scale,
        "set5_psnr": set5_psnr,
        "set14_psnr": set14_psnr,
    }

    # last.pth 每个 epoch 都会覆盖保存，表示“最后一次训练得到的模型”。
    torch.save(checkpoint, save_dir / "last.pth")

    if mean_psnr > best_score:
        best_score = mean_psnr

    # best.pth 只在验证效果变好时保存。
    torch.save(checkpoint, save_dir / "best.pth")
    print(f"Saved best model: mean PSNR = {best_score:.2f} dB")

def parse_args():
    # 创建命令行参数解析器。
    parser = argparse.ArgumentParser()

    # 数据路径参数。
    parser.add_argument("--train-dir", type=str, default="data/DIV2K/DIV2K_train_HR")
    parser.add_argument("--set5-dir", type=str, default="data/benchmark/Set5/HR")
    parser.add_argument("--set14-dir", type=str, default="data/benchmark/Set14/HR")
    parser.add_argument("--save-dir", type=str, default="outputs/checkpoints")

    # 训练超参数。
    parser.add_argument("--scale", type=int, default=2)
    parser.add_argument("--patch-size", type=int, default=96)
    parser.add_argument("--samples-per-epoch", type=int, default=2000)
    parser.add_argument("--batch-size", type=int, default=16)
    parser.add_argument("--epochs", type=int, default=50)
    parser.add_argument("--lr", type=float, default=1e-4)

    # 加上 --cpu 后，强制使用 CPU 训练或测试。
    parser.add_argument("--cpu", action="store_true")

    # parse_args() 会读取命令行输入，并返回 args 对象。
    return parser.parse_args()

if __name__ == "__main__":
    # 只有直接运行 python src/train.py 时，下面两行才会执行。
    # 如果这个文件被其他 Python 文件 import，则不会自动开始训练。

```

```
args = parse_args()
train(args)
```

重点理解：

- `train.py` 是整个实验的核心脚本。
- 每个 batch 的训练顺序是：前向传播、计算损失、清空梯度、反向传播、更新参数。
- 每个 epoch 后都会在 Set5 和 Set14 上验证。
- `best.pth` 保存验证 PSNR 最好的模型。

## 6.5 测试 test.py

文件路径：

```
src/test.py
```

完整代码：

```
# argparse 用于读取测试脚本的命令行参数。
import argparse

# Path 用于处理输出目录和文件路径。
from pathlib import Path

import torch

# DataLoader 用于按顺序读取测试集中的图像。
from torch.utils.data import DataLoader

# tqdm 用于显示测试进度。
from tqdm import tqdm

from dataset import SRDataset
from model import SRCNN
from utils import calc_psnr, save_compare_image, tensor_to_image

@torch.no_grad()
def test_dataset(model, dataset_name, hr_dir, args, device):
    """
    在一个测试集上测试模型，并保存对比图。

    dataset_name:
        数据集名称，例如 Set5 或 Set14。

    hr_dir:
        当前测试集的 HR 图像目录。
    """
```

```

# 创建测试集。
# train=False 表示不做随机裁剪，而是使用完整 HR 图像进行测试。
dataset = SRDataset(hr_dir, scale=args.scale, train=False)

# 测试图像尺寸可能不同，因此 batch_size 固定为 1。
# shuffle=False 表示按文件名顺序测试，方便和输出结果对应。
loader = DataLoader(dataset, batch_size=1, shuffle=False, num_workers=0)

# 每个测试集单独保存到一个子目录中，例如 outputs/results/Set5。
output_dir = Path(args.output_dir) / dataset_name
output_dir.mkdir(parents=True, exist_ok=True)

# 分别累加 Bicubic 和 SRCNN 的 PSNR，最后再求平均。
total_bicubic_psnr = 0.0
total_srcnn_psnr = 0.0

# 切换到测试模式。
model.eval()

for batch in tqdm(loader, desc=f"Testing {dataset_name}"):
    # lr 是 Bicubic 放大后的图像，是模型输入。
    # hr 是真实高分辨率图像，是评价指标中的参考图。
    lr = batch["lr"].to(device)
    hr = batch["hr"].to(device)

    # DataLoader 会把字符串 name 组织成列表或元组。
    # batch_size=1 时，取第 0 个就是当前图片名。
    name = batch["name"][0]

    # 使用训练好的 SRCNN 生成超分辨率图像。
    sr = model(lr)

    # Bicubic PSNR 衡量“只用插值”的效果。
    bicubic_psnr = calc_psnr(lr, hr)

    # SRCNN PSNR 衡量“插值 + 神经网络修复”的效果。
    srcnn_psnr = calc_psnr(sr, hr)

    total_bicubic_psnr += bicubic_psnr
    total_srcnn_psnr += srcnn_psnr

    # lr、sr、hr 的 batch 维度都是 1。
    # lr[0] 表示取出这一张图本身，形状从 1 x 3 x H x W 变成 3 x H x W。
    lr_image = tensor_to_image(lr[0])
    sr_image = tensor_to_image(sr[0])
    hr_image = tensor_to_image(hr[0])

    # 保存横向对比图，方便肉眼观察模型是否恢复了更多细节。
    save_compare_image(
        lr_image,
        sr_image,
        hr_image,

```

```

        output_dir / f"{name}_compare.png",
    )

    # len(dataset) 就是当前测试集的图像数量。
    count = len(dataset)

    # 计算平均 PSNR。
    avg_bicubic = total_bicubic_psnr / count
    avg_srcnn = total_srcnn_psnr / count

    print(f"{dataset_name} Bicubic PSNR: {avg_bicubic:.2f} dB")
    print(f"{dataset_name} SRCNN PSNR: {avg_srcnn:.2f} dB")
    print(f"{dataset_name} Improvement: {avg_srcnn - avg_bicubic:.2f} dB")

    return avg_bicubic, avg_srcnn

def main(args):
    # 自动选择 CPU 或 GPU。
    device = torch.device("cuda" if torch.cuda.is_available() and not args.cpu else "cpu")
    print(f"Using device: {device}")

    # 创建一个和训练时相同结构的 SRCNN。
    model = SRCNN().to(device)

    # torch.load 读取训练时保存的 checkpoint。
    # map_location=device 表示无论模型原来在哪个设备保存，都加载到当前 device。
    checkpoint = torch.load(args.checkpoint, map_location=device)

    # 把 checkpoint 中的模型参数加载到当前 model 中。
    model.load_state_dict(checkpoint["model"])

    # 分别测试 Set5 和 Set14。
    test_dataset(model, "Set5", args.set5_dir, args, device)
    test_dataset(model, "Set14", args.set14_dir, args, device)

def parse_args():
    parser = argparse.ArgumentParser()

    # checkpoint 是训练得到的模型文件。
    parser.add_argument("--checkpoint", type=str, default="outputs/checkpoints/best.pth")

    # 两个测试集路径。
    parser.add_argument("--set5-dir", type=str, default="data/benchmark/Set5/HR")
    parser.add_argument("--set14-dir", type=str, default="data/benchmark/Set14/HR")

    # 结果图保存目录。
    parser.add_argument("--output-dir", type=str, default="outputs/results")

    # 放大倍数必须和训练时保持一致。
    parser.add_argument("--scale", type=int, default=2)

```

```

# 加上 --cpu 后强制使用 CPU。
parser.add_argument("--cpu", action="store_true")

return parser.parse_args()

if __name__ == "__main__":
    # 解析参数并开始测试。
    args = parse_args()
    main(args)

```

重点理解：

- `test.py` 不会训练模型，只会加载已经训练好的 `best.pth`。
- 它会分别测试 Set5 和 Set14。
- 它会同时输出 Bicubic 和 SRCNN 的 PSNR，便于比较模型是否真的有效。
- 每张图都会保存一张 `Bicubic | SRCNN | HR` 对比图。

## 6.6 单张图像推理 `infer.py`（需要大家自己补充，作为实验报告的一部分，见10.4）

文件路径：

```
src/infer.py
```

部分代码：

```

# argparse 用于读取单张图像推理时的命令行参数。
import argparse

# Path 用于创建输出目录和处理输出文件路径。
from pathlib import Path

import torch

# Image 用于打开学生自己拍摄或准备的输入图像。
from PIL import Image

from model import SRCNN
from utils import image_to_tensor, tensor_to_image

@torch.no_grad()
def main(args):
    # 这里作为实验报告的补充任务，由同学们自己完成。
    # 建议按下面的顺序写：
    #

```

```

# 1. 选择 device:
#     如果有 GPU 且没有传入 --cpu, 就使用 cuda, 否则使用 cpu。
#
# 2. 读取输入图片:
#     使用 Image.open(args.input).convert("RGB") 打开图像。
#
# 3. Bicubic 放大 (如果是edsr则不需要先放大):
#     根据 args.scale 计算放大后的宽和高,
#     再用 image.resize(..., Image.BICUBIC) 得到模型输入图。
#
# 4. 转成 Tensor:
#     使用 image_to_tensor, 把图像从 H x W x C 转成 3 x H x W,
#     再用 unsqueeze(0) 增加 batch 维度, 变成 1 x 3 x H x W。
#
# 5. 加载模型:
#     创建 SRCNN().to(device), 再用 torch.load 读取 checkpoint,
#     最后调用 model.load_state_dict(checkpoint["model"])。
#
# 6. 前向推理:
#     sr = model(lr), 得到 SRCNN 输出。
#
# 7. 保存结果:
#     使用 tensor_to_image(sr[0]) 转回 PIL 图像,
#     创建输出目录, 并保存到 args.output。
#
# 这部分暂时抛出错误, 提醒大家这里还没有补全, 补全后可以删除下面这行代码
raise NotImplementedError("请按照注释提示补充 infer.py 的单张图像推理代码")

```

```

def parse_args():
    parser = argparse.ArgumentParser()

    # 输入图片路径, required=True 表示运行时必须提供。
    parser.add_argument("--input", type=str, required=True)

    # 输出图片保存路径。
    parser.add_argument("--output", type=str, default="outputs/results/infer_srcnn.png")

    # 已训练好的 SRCNN 模型路径。
    parser.add_argument("--checkpoint", type=str, default="outputs/checkpoints/best.pth")

    # 放大倍数, 例如 scale=2 表示把图片宽高都放大 2 倍。
    parser.add_argument("--scale", type=int, default=2)

    # 加上 --cpu 后强制使用 CPU 推理。
    parser.add_argument("--cpu", action="store_true")

    return parser.parse_args()

```

```

if __name__ == "__main__":
    # 解析命令行参数, 并进入 main 函数。

```

```
args = parse_args()
main(args)
```

重点理解：

- `infer.py` 用于处理一张新的图片。
- 它会先把输入图像用 Bicubic 放大 `scale` 倍。
- 然后再使用 SRCNN 对放大后的图像进行修复。

## 7. 训练与测试命令

### 7.1 训练模型

在项目根目录执行：

```
python src/train.py \
  --train-dir data/DIV2K/DIV2K_train_HR \
  --set5-dir data/benchmark/Set5/HR \
  --set14-dir data/benchmark/Set14/HR \
  --scale 4 \
  --patch-size 128 \
  --samples-per-epoch 2000 \
  --batch-size 32 \
  --epochs 50 \
  --lr 1e-4
```

### 7.2 训练日志示例

点击vscode右下角的加号，新建一个终端，输入下面的命令来查看显卡的使用情况：

```
watch -n 1 nvidia-smi
```

训练时会看到类似输出：

```
Using device: cuda
Epoch 1/50: 100%|██████████| 125/125 [00:12<00:00, loss=0.003821]
Epoch 001 | loss=0.005614 | Set5=28.20 dB | Set14=26.71 dB
Saved best model: mean PSNR = 27.46 dB
```

说明：

- `loss` 是训练集上的 MSE 损失。
- `Set5` 和 `Set14` 是当前模型在测试集上的 PSNR。
- 随着训练进行，loss 通常会下降，PSNR 通常会上升。

### 7.3 测试模型

训练完成后，运行：

```
python src/test.py \  
  --checkpoint outputs/checkpoints/best.pth \  
  --set5-dir data/benchmark/Set5/HR \  
  --set14-dir data/benchmark/Set14/HR \  
  --output-dir outputs/results \  
  --scale 4
```

如果使用 CPU：

```
python src/test.py \  
  --checkpoint outputs/checkpoints/best.pth \  
  --set5-dir data/benchmark/Set5/HR \  
  --set14-dir data/benchmark/Set14/HR \  
  --output-dir outputs/results \  
  --scale 4 \  
  --cpu
```

测试输出示例：

```
Set5 Bicubic PSNR: 28.10 dB  
Set5 SRCNN PSNR: 28.78 dB  
Set5 Improvement: 0.68 dB  
  
Set14 Bicubic PSNR: 26.40 dB  
Set14 SRCNN PSNR: 26.92 dB  
Set14 Improvement: 0.52 dB
```

注意：上面的数字只是示例，不是固定结果。实际数值会受到训练轮数、硬件、随机初始化和数据整理方式影响。

## 7.4 查看输出图片

测试完成后，可以查看：

```
outputs/results/Set5/  
outputs/results/Set14/
```

每张对比图包含：

```
Bicubic | SRCNN | HR
```

观察时重点看：

1. SRCNN 是否比 Bicubic 更锐利。
2. 边缘是否更清楚。
3. 纹理是否更接近 HR。



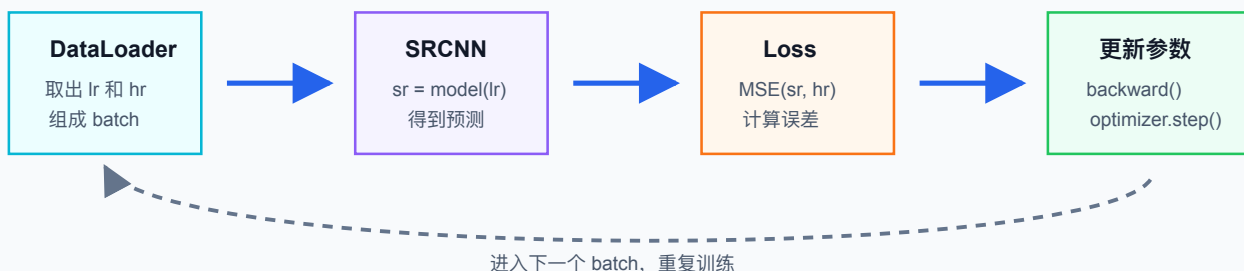
4. 是否出现明显伪影。

## 8. 代码逻辑总览

### 8.1 数据流

```
HR 图像
|
训练时随机裁剪 patch
|
缩小 scale 倍, 得到 LR
|
Bicubic 放大回 HR 尺寸, 得到模型输入
|
SRCNN 输出 SR
|
计算 MSE(SR, HR)
|
反向传播并更新模型参数
```

**训练流程：每个 batch 都重复一次前向传播和反向传播**



这张图对应训练代码中的核心几行：`model(lr)`、`loss_fn(sr, hr)`、`loss.backward()` 和 `optimizer.step()`。理解这张图后，再阅读 `train.py` 会更容易。

### 8.2 训练循环

训练循环的核心代码是：

```
sr = model(lr)
loss = loss_fn(sr, hr)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

这 5 行对应深度学习训练的核心流程：

1. 模型预测。

2. 计算误差。
3. 清空旧梯度。
4. 反向传播计算新梯度。
5. 更新参数。

## 8.3 测试循环

测试阶段不更新模型参数，只做：

```
sr = model(lr)
psnr = calc_psnr(sr, hr)
```

测试阶段的目标是评估模型效果，而不是继续学习。

## 9. 扩展实验：使用 PixelShuffle 版 EDSR

前面的实验使用的是 SRCNN。SRCNN 的典型流程是：先用 Bicubic 把低分辨率图像放大到目标尺寸，再让卷积网络在这个放大图上修复细节。这个思路很适合入门，但它把“放大尺寸”交给了固定插值算法，网络主要负责后处理。

EDSR 的全称是 Enhanced Deep Residual Networks for Single Image Super-Resolution。更接近官方 EDSR 的做法是：直接输入小尺寸 LR 图像，让网络自己在内部完成上采样。本节将 EDSR 改成这种形式：

小尺寸 LR 图像 -> 特征提取 -> 多个残差块 -> PixelShuffle 上采样 -> SR 输出图像

也就是说，本节的 EDSR 不再使用“Bicubic 上采样 + 网络修复”的形式。Bicubic 仍然会保留在测试代码里，但它只作为基线方法，用来和 EDSR 的输出做 PSNR 与视觉效果比较。

### PixelShuffle 版 EDSR：直接输入小尺寸 LR，由网络内部完成上采样



关键区别：Bicubic 只作为测试基线；EDSR 的输入是小尺寸 LR，放大尺寸由 PixelShuffle 完成。

scale=4 时常见做法是连续两次 PixelShuffle(2)，每次让宽和高扩大 2 倍。

这张图展示了本节使用的 EDSR 风格网络。输入是小尺寸 LR 图像，网络先在低分辨率特征空间中提取和修复纹理，再通过 PixelShuffle 把特征图放大到目标尺寸，最后输出 SR 图像。

## 9.1 EDSR 的核心思想

EDSR 中最重要的结构有两个：

1. 残差块：让深层网络更容易训练。

2. PixelShuffle 上采样：让网络自己学习如何放大图像。

这里需要特别说明：PixelShuffle 是网络内部的上采样方法，不是下采样方法。训练时的小尺寸 LR 图像仍然需要先从 HR 图像生成，本实验采用 Bicubic 下采样模拟低分辨率图像；随后 EDSR 再学习如何从这个 LR 图像重建 HR 图像。

残差块可以写成：

```
输出 = 输入 + F(输入)
```

其中 `F(输入)` 表示卷积层学习到的变化量。这样设计可以让网络在保留已有特征的基础上补充新信息，梯度也更容易在深层网络中传播。

PixelShuffle 的作用是把“通道维度中的信息”重新排列到“空间维度”。以 2 倍上采样为例：

```
卷积输出: B x (4C) x H x W  
PixelShuffle(2) 后: B x C x (2H) x (2W)
```

也就是说，PixelShuffle 不像 Bicubic 那样使用固定数学公式插值，而是先通过卷积学习出更多通道，再把这些通道重新排列成更大的图像。对于 4 倍超分辨率，本实验使用两次 `PixelShuffle(2)`：

```
H x W -> 2H x 2W -> 4H x 4W
```

## 9.2 新增文件说明

本节不修改前面的 SRCNN 代码，而是新增 4 个文件：

```
src/  
├─ dataset_edsr.py  
├─ model_edsr.py  
├─ train_edsr.py  
└─ test_edsr.py
```

需要单独写 `dataset_edsr.py` 的原因是：SRCNN 和 EDSR 的输入尺寸不同。

- SRCNN 的输入是 Bicubic 放大后的图像，尺寸已经等于 HR。
- EDSR 的输入是小尺寸 LR 图像，尺寸是 HR 的 `1 / scale`。
- EDSR 的输出才是 HR 尺寸图像。

这样写可以让前面的 SRCNN 实验继续正常运行，也能让 EDSR 更接近真实论文和常见代码库中的实现方式。

---

## 9.3 EDSR 数据集代码 dataset\_edsr.py

文件路径：

```
src/dataset_edsr.py
```

完整代码：

```
import random

from PIL import Image
from torch.utils.data import Dataset

from utils import image_to_tensor, list_image_files

class EDSRDataset(Dataset):
    """
    EDSR 使用的数据集。

    和 SRCNN 的 Dataset 相比，最大区别是：
        SRCNN 返回的是 Bicubic 放大后的 lr_up，尺寸和 HR 相同。
        EDSR 返回的是小尺寸 lr，尺寸比 HR 小 scale 倍。

    因此：
        lr 的形状是 3 x (H / scale) x (W / scale)
        hr 的形状是 3 x H x W

    EDSR 模型会在网络内部使用 PixelShuffle 把 lr 放大成 SR。
    """

    def __init__(self, hr_dir, scale=4, patch_size=128, train=True,
samples_per_epoch=2000):
        """
        参数：
            hr_dir: HR 图像所在文件夹。
            scale: 超分辨率放大倍数，例如 4 表示 x4 超分辨率。
            patch_size: 从 HR 图像中裁剪出的训练 patch 大小。
            train: True 表示训练模式，False 表示测试模式。
            samples_per_epoch: 每个 epoch 随机采样多少个训练 patch。
        """
        self.image_files = list_image_files(hr_dir)
        self.scale = scale
        self.patch_size = patch_size
        self.train = train
        self.samples_per_epoch = samples_per_epoch

        if len(self.image_files) == 0:
            raise RuntimeError(f"No image files found in {hr_dir}")

        # HR patch 必须能被 scale 整除。
        # 例如 scale=4 时, patch_size=128 可以生成 32 x 32 的 LR patch。
        if patch_size % scale != 0:
            raise ValueError("patch_size must be divisible by scale")

        # 为了加快训练，仍然把所有 HR 图像一次性读入内存。
        self.images = []
        print(f"Loading {len(self.image_files)} HR images into memory for EDSR...")
```

```

for path in self.image_files:
    image = Image.open(path).convert("RGB")
    image = self.crop_to_multiple(image)
    self.images.append(
        {
            "name": path.stem,
            "image": image,
        }
    )

print("Finished loading EDSR images.")

def __len__(self):
    """
    训练时返回 samples_per_epoch。
    测试时返回真实图像数量。
    """
    if self.train:
        return self.samples_per_epoch

    return len(self.images)

def crop_to_multiple(self, image):
    """
    把图像裁剪到宽和高都能被 scale 整除。

    如果不这样做, LR 放大 scale 倍后, 可能和 HR 尺寸对不上。
    """
    width, height = image.size
    width = width - width % self.scale
    height = height - height % self.scale
    return image.crop((0, 0, width, height))

def random_crop(self, image):
    """
    从 HR 图像中随机裁剪一个 patch。

    注意:
        patch_size 指的是 HR patch 的大小。
        LR patch 的大小会自动变成 patch_size / scale。
    """
    width, height = image.size

    if width < self.patch_size or height < self.patch_size:
        image = image.resize((self.patch_size, self.patch_size), Image.BICUBIC)
        return image

    left = random.randint(0, width - self.patch_size)
    top = random.randint(0, height - self.patch_size)
    right = left + self.patch_size
    bottom = top + self.patch_size

```

```

        return image.crop((left, top, right, bottom))

def make_lr(self, hr):
    """
    根据 HR 图像生成小尺寸 LR 图像。

    这里只做一次下采样：
        HR -> LR

    不再做第二次 Bicubic 放大，因为 EDSR 会在网络内部完成上采样。

    注意：
        这里的 Bicubic 是用来构造训练输入的退化过程。
        PixelShuffle 只出现在 EDSR 网络内部，用于 LR -> SR 的上采样。
    """
    width, height = hr.size

    lr_width = width // self.scale
    lr_height = height // self.scale

    lr = hr.resize((lr_width, lr_height), Image.BICUBIC)

    return lr

def __getitem__(self, index):
    """
    返回一个样本。

    返回字典：
        lr:    小尺寸低分辨率图像 Tensor
        hr:    真实高分辨率图像 Tensor
        name:  图像文件名
    """
    item = self.images[index % len(self.images)]

    # copy() 可以避免随机裁剪影响内存中的原始图像。
    hr = item["image"].copy()

    if self.train:
        hr = self.random_crop(hr)

    # 生成小尺寸 LR。
    lr = self.make_lr(hr)

    return {
        "lr": image_to_tensor(lr),
        "hr": image_to_tensor(hr),
        "name": item["name"],
    }

```

理解这一段时，请特别注意：`make_lr` 只负责把 HR 缩小成 LR，不会再把 LR 放大回 HR 尺寸。上采样现在交给 EDSR 网络内部完成。

## 9.4 EDSR 模型代码 `model_edsr.py`

文件路径：

```
src/model_edsr.py
```

完整代码：

```
import torch.nn as nn

class ResidualBlock(nn.Module):
    """
    EDSR 中的残差块。

    EDSR 的一个重要特点是去掉 BatchNorm。
    对图像超分辨率任务来说，去掉 BatchNorm 往往可以减少显存占用，
    也有助于保留图像像素值的细节范围。
    """

    def __init__(self, num_features, res_scale=0.1):
        super().__init__()

        self.res_scale = res_scale

        self.block = nn.Sequential(
            # 第一层 3 x 3 卷积，保持特征图宽高不变。
            nn.Conv2d(num_features, num_features, kernel_size=3, padding=1),

            # ReLU 提供非线性表达能力。
            nn.ReLU(inplace=True),

            # 第二层 3 x 3 卷积。
            # 后面要和输入做残差相加，因此输出通道仍然是 num_features。
            nn.Conv2d(num_features, num_features, kernel_size=3, padding=1),
        )

    def forward(self, x):
        # residual 是残差分支学习到的修正量。
        residual = self.block(x)

        # res_scale 可以让深层网络训练更稳定。
        residual = residual * self.res_scale

        # 残差连接：输出 = 原特征 + 修正量。
        return x + residual
```

```

class Upsampler(nn.Sequential):
    """
    PixelShuffle 上采样模块。

    这个模块把低分辨率特征图放大到高分辨率特征图。
    支持 scale=2、3、4、8。
    """

    def __init__(self, scale, num_features):
        layers = []

        if scale in [2, 4, 8]:
            # scale=2 需要 1 次 PixelShuffle(2)。
            # scale=4 需要 2 次 PixelShuffle(2)。
            # scale=8 需要 3 次 PixelShuffle(2)。
            num_upsample = {2: 1, 4: 2, 8: 3}[scale]

            for _ in range(num_upsample):
                layers.append(
                    nn.Conv2d(
                        in_channels=num_features,
                        out_channels=num_features * 4,
                        kernel_size=3,
                        padding=1,
                    )
                )

                # PixelShuffle(2) 会把通道数缩小 4 倍,
                # 同时把宽和高都放大 2 倍。
                layers.append(nn.PixelShuffle(2))

        elif scale == 3:
            layers.append(
                nn.Conv2d(
                    in_channels=num_features,
                    out_channels=num_features * 9,
                    kernel_size=3,
                    padding=1,
                )
            )

            # PixelShuffle(3) 会把通道数缩小 9 倍,
            # 同时把宽和高都放大 3 倍。
            layers.append(nn.PixelShuffle(3))

        else:
            raise ValueError("scale must be 2, 3, 4, or 8")

        super().__init__(*layers)

```



```

class EDSR(nn.Module):
    """
    PixelShuffle 版 EDSR。

    输入：
        B x 3 x h x w

    输出：
        B x 3 x (h * scale) x (w * scale)

    和前面的 SRCNN 不同：
        SRCNN 的输入已经是放大后的图像。
        EDSR 的输入是小尺寸 LR 图像，网络内部自己完成放大。
    """

    def __init__(self, scale=4, num_features=64, num_blocks=8, res_scale=0.1):
        super().__init__()

        self.scale = scale

        # Head: 把 RGB 图像映射到特征空间。
        self.head = nn.Conv2d(
            in_channels=3,
            out_channels=num_features,
            kernel_size=3,
            padding=1,
        )

        # Body: 多个残差块串联。
        body_layers = []
        for _ in range(num_blocks):
            body_layers.append(
                ResidualBlock(
                    num_features=num_features,
                    res_scale=res_scale,
                )
            )

        # Body 末尾的卷积用于整合所有残差块后的特征。
        body_layers.append(
            nn.Conv2d(
                in_channels=num_features,
                out_channels=num_features,
                kernel_size=3,
                padding=1,
            )
        )

        self.body = nn.Sequential(*body_layers)

        # Upsampler: 使用 PixelShuffle 完成上采样。
        self.upsampler = Upsampler(scale=scale, num_features=num_features)

```

```

# Tail: 把上采样后的特征图映射回 RGB 图像。
self.tail = nn.Conv2d(
    in_channels=num_features,
    out_channels=3,
    kernel_size=3,
    padding=1,
)

def forward(self, x):
    """
    前向传播。

    x 是小尺寸 LR 图像。
    输出 sr 是放大 scale 倍后的 SR 图像。
    """
    # 提取浅层特征。
    shallow_feature = self.head(x)

    # 通过残差块学习深层特征。
    deep_feature = self.body(shallow_feature)

    # EDSR 的长残差连接发生在特征空间中。
    # 这里 shallow_feature 和 deep_feature 尺寸相同，可以直接相加。
    feature = shallow_feature + deep_feature

    # PixelShuffle 上采样，把特征图放大到目标尺寸。
    up_feature = self.upsampler(feature)

    # 输出 RGB 图像。
    sr = self.tail(up_feature)

    return sr

```

代码理解重点：

1. `ResidualBlock` 负责在低分辨率特征空间中学习细节。
2. `Upsampler` 负责使用 `PixelShuffle` 放大特征图。
3. `scale=4` 时，代码会连续执行两次 `PixelShuffle(2)`。
4. EDSR 的输入是小尺寸 LR，输出才是 HR 尺寸。
5. 因为 LR 和 HR 尺寸不同，不能再像前一版那样直接写 `return x + residual_image`。

## 9.5 EDSR 训练代码 train\_edsr.py

文件路径：

```
src/train_edsr.py
```

完整代码：

```
import argparse
from pathlib import Path

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from tqdm import tqdm

from dataset_edsr import EDSRDataset
from model_edsr import EDSR
from utils import calc_psnr

@torch.no_grad()
def evaluate(model, data_loader, device):
    """
    在测试集上计算 EDSR 的平均 PSNR。

    data_loader 返回的小尺寸 LR 会直接输入模型。
    模型输出的 SR 尺寸应该和 HR 一致。
    """
    model.eval()

    total_psnr = 0.0
    image_count = 0

    for batch in data_loader:
        lr = batch["lr"].to(device)
        hr = batch["hr"].to(device)

        # EDSR 内部会使用 PixelShuffle 上采样。
        sr = model(lr)

        psnr = calc_psnr(sr, hr)
        total_psnr += psnr
        image_count += 1

    return total_psnr / image_count

def train(args):
    """
    EDSR 训练主函数。
```

主要流程：

1. 使用 EDSRDataset 读取 HR，并生成小尺寸 LR。
2. 使用 EDSR 模型从 LR 预测 SR。
3. 使用 SR 和 HR 计算损失。
4. 每个 epoch 后在 Set5 和 Set14 上验证。

```

    5. 保存 last.pth 和 best.pth。
"""
device = torch.device("cuda" if torch.cuda.is_available() and not args.cpu else "cpu")
print(f"Using device: {device}")

train_set = EDSRDataset(
    hr_dir=args.train_dir,
    scale=args.scale,
    patch_size=args.patch_size,
    train=True,
    samples_per_epoch=args.samples_per_epoch,
)

set5 = EDSRDataset(args.set5_dir, scale=args.scale, train=False)
set14 = EDSRDataset(args.set14_dir, scale=args.scale, train=False)

train_loader = DataLoader(
    train_set,
    batch_size=args.batch_size,
    shuffle=True,
    num_workers=0,
)

set5_loader = DataLoader(set5, batch_size=1, shuffle=False, num_workers=0)
set14_loader = DataLoader(set14, batch_size=1, shuffle=False, num_workers=0)

model = EDSR(
    scale=args.scale,
    num_features=args.num_features,
    num_blocks=args.num_blocks,
    res_scale=args.res_scale,
).to(device)

# EDSR 常见实现中经常使用 L1Loss。
# L1Loss 计算的是像素绝对误差，比 MSELoss 对少量大误差更不敏感。
loss_fn = nn.L1Loss()

optimizer = optim.Adam(model.parameters(), lr=args.lr)

save_dir = Path(args.save_dir)
save_dir.mkdir(parents=True, exist_ok=True)

best_score = 0.0

for epoch in range(1, args.epochs + 1):
    model.train()
    total_loss = 0.0

    progress = tqdm(train_loader, desc=f"EDSR Epoch {epoch}/{args.epochs}")

    for batch in progress:
        # lr 是小尺寸图像，hr 是对应的高分辨率标签。

```

```

    lr = batch["lr"].to(device)
    hr = batch["hr"].to(device)

    # 模型内部完成上采样, 输出 SR。
    sr = model(lr)

    # SR 和 HR 尺寸相同, 可以直接计算损失。
    loss = loss_fn(sr, hr)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    total_loss += loss.item()
    progress.set_postfix(loss=f"{loss.item():.6f}")

avg_loss = total_loss / len(train_loader)

set5_psnr = evaluate(model, set5_loader, device)
set14_psnr = evaluate(model, set14_loader, device)
mean_psnr = (set5_psnr + set14_psnr) / 2.0

print(
    f"EDSR Epoch {epoch:03d} | "
    f"loss={avg_loss:.6f} | "
    f"Set5={set5_psnr:.2f} dB | "
    f"Set14={set14_psnr:.2f} dB"
)

checkpoint = {
    "model": model.state_dict(),
    "epoch": epoch,
    "scale": args.scale,
    "num_features": args.num_features,
    "num_blocks": args.num_blocks,
    "res_scale": args.res_scale,
    "set5_psnr": set5_psnr,
    "set14_psnr": set14_psnr,
}

torch.save(checkpoint, save_dir / "last.pth")

if mean_psnr > best_score:
    best_score = mean_psnr
    torch.save(checkpoint, save_dir / "best.pth")
    print(f"Saved best EDSR model: mean PSNR = {best_score:.2f} dB")

def parse_args():
    parser = argparse.ArgumentParser()

    parser.add_argument("--train-dir", type=str, default="data/DIV2K/DIV2K_train_HR")

```

```

parser.add_argument("--set5-dir", type=str, default="data/benchmark/Set5/HR")
parser.add_argument("--set14-dir", type=str, default="data/benchmark/Set14/HR")
parser.add_argument("--save-dir", type=str, default="outputs/checkpoints_edsr")

parser.add_argument("--scale", type=int, default=4)
parser.add_argument("--patch-size", type=int, default=128)
parser.add_argument("--samples-per-epoch", type=int, default=2000)
parser.add_argument("--batch-size", type=int, default=16)
parser.add_argument("--epochs", type=int, default=50)
parser.add_argument("--lr", type=float, default=1e-4)

parser.add_argument("--num-blocks", type=int, default=8)
parser.add_argument("--num-features", type=int, default=64)
parser.add_argument("--res-scale", type=float, default=0.1)

parser.add_argument("--cpu", action="store_true")

return parser.parse_args()

if __name__ == "__main__":
    args = parse_args()
    train(args)

```

与 SRCNN 训练代码相比，最重要的变化是：

1. 数据集从 `SRDataset` 改成 `EDSRDataset`。
2. 模型输入从 Bicubic 放大图改成小尺寸 LR。
3. 模型内部通过 `PixelShuffle` 完成上采样。
4. 损失函数从 `MSELoss` 改成更常用于 EDSR 的 `L1Loss`。

## 9.6 EDSR 测试代码 test\_edsr.py

文件路径：

```
src/test_edsr.py
```

完整代码：

```

import argparse
from pathlib import Path

import torch
import torch.nn.functional as F
from PIL import Image, ImageDraw
from torch.utils.data import DataLoader
from tqdm import tqdm

```

```

from dataset_edsr import EDSRDataset
from model_edsr import EDSR
from utils import calc_psnr, tensor_to_image

def save_edsr_compare_image(bicubic_image, sr_image, hr_image, save_path):
    """
    保存 EDSR 测试对比图。

    对比图内容：
        Bicubic | EDSR | HR

    注意：
        这里的 Bicubic 是由小尺寸 LR 放大到 HR 尺寸得到的基线结果。
        它不是 EDSR 的输入，只是用来对比。
    """
    width, height = hr_image.size
    label_height = 28

    canvas = Image.new("RGB", (width * 3, height + label_height), "white")

    canvas.paste(bicubic_image, (0, label_height))
    canvas.paste(sr_image, (width, label_height))
    canvas.paste(hr_image, (width * 2, label_height))

    draw = ImageDraw.Draw(canvas)
    draw.text((10, 8), "Bicubic", fill=(0, 0, 0))
    draw.text((width + 10, 8), "EDSR", fill=(0, 0, 0))
    draw.text((width * 2 + 10, 8), "HR", fill=(0, 0, 0))

    save_path = Path(save_path)
    save_path.parent.mkdir(parents=True, exist_ok=True)
    canvas.save(save_path)

@torch.no_grad()
def test_dataset(model, dataset_name, hr_dir, args, device):
    """
    在一个测试集上测试 EDSR。

    测试时会同时计算：
        1. Bicubic 基线 PSNR。
        2. EDSR 输出 PSNR。
    """
    dataset = EDSRDataset(hr_dir, scale=args.scale, train=False)
    loader = DataLoader(dataset, batch_size=1, shuffle=False, num_workers=0)

    output_dir = Path(args.output_dir) / dataset_name
    output_dir.mkdir(parents=True, exist_ok=True)

    total_bicubic_psnr = 0.0
    total_edsr_psnr = 0.0

```

```

model.eval()

for batch in tqdm(loader, desc=f"Testing EDSR on {dataset_name}"):
    # lr 是小尺寸图像, hr 是高分辨率参考图。
    lr = batch["lr"].to(device)
    hr = batch["hr"].to(device)
    name = batch["name"][0]

    # EDSR 内部使用 PixelShuffle 得到 SR。
    sr = model(lr)

    # 为了得到 Bicubic 基线, 需要把同一张小尺寸 LR 放大到 HR 尺寸。
    # F.interpolate 是 PyTorch 中常用的张量缩放函数。
    bicubic = F.interpolate(
        lr,
        size=hr.shape[-2:],
        mode="bicubic",
        align_corners=False,
    )

    bicubic_psnr = calc_psnr(bicubic, hr)
    edsr_psnr = calc_psnr(sr, hr)

    total_bicubic_psnr += bicubic_psnr
    total_edsr_psnr += edsr_psnr

    bicubic_image = tensor_to_image(bicubic[0])
    sr_image = tensor_to_image(sr[0])
    hr_image = tensor_to_image(hr[0])

    save_edsr_compare_image(
        bicubic_image,
        sr_image,
        hr_image,
        output_dir / f"{name}_edsr_compare.png",
    )

count = len(dataset)
avg_bicubic = total_bicubic_psnr / count
avg_edsr = total_edsr_psnr / count

print(f"{dataset_name} Bicubic PSNR: {avg_bicubic:.2f} dB")
print(f"{dataset_name} EDSR PSNR: {avg_edsr:.2f} dB")
print(f"{dataset_name} Improvement: {avg_edsr - avg_bicubic:.2f} dB")

return avg_bicubic, avg_edsr

def main(args):
    device = torch.device("cuda" if torch.cuda.is_available() and not args.cpu else "cpu")
    print(f"Using device: {device}")

```



```

checkpoint = torch.load(args.checkpoint, map_location=device)

# 测试时的数据 scale 必须和训练时一致。
# 如果 checkpoint 里保存了 scale, 就优先使用 checkpoint 中的值。
scale = checkpoint.get("scale", args.scale)
args.scale = scale

model = EDSR(
    scale=scale,
    num_features=checkpoint.get("num_features", args.num_features),
    num_blocks=checkpoint.get("num_blocks", args.num_blocks),
    res_scale=checkpoint.get("res_scale", args.res_scale),
).to(device)

model.load_state_dict(checkpoint["model"])

test_dataset(model, "Set5", args.set5_dir, args, device)
test_dataset(model, "Set14", args.set14_dir, args, device)

def parse_args():
    parser = argparse.ArgumentParser()

    parser.add_argument("--checkpoint", type=str,
default="outputs/checkpoints_edsr/best.pth")
    parser.add_argument("--set5-dir", type=str, default="data/benchmark/Set5/HR")
    parser.add_argument("--set14-dir", type=str, default="data/benchmark/Set14/HR")
    parser.add_argument("--output-dir", type=str, default="outputs/results_edsr")

    parser.add_argument("--scale", type=int, default=4)
    parser.add_argument("--num-blocks", type=int, default=8)
    parser.add_argument("--num-features", type=int, default=64)
    parser.add_argument("--res-scale", type=float, default=0.1)

    parser.add_argument("--cpu", action="store_true")

    return parser.parse_args()

if __name__ == "__main__":
    args = parse_args()
    main(args)

```

测试代码中最容易混淆的是 `bicubic` 变量：它不是模型输入，而是为了和 EDSR 比较，临时把 LR 插值到 HR 尺寸得到的传统方法结果。

## 9.7 训练与测试 EDSR

在项目根目录执行训练：

```
python src/train_edsr.py \  
  --train-dir data/DIV2K/DIV2K_train_HR \  
  --set5-dir data/benchmark/Set5/HR \  
  --set14-dir data/benchmark/Set14/HR \  
  --scale 4 \  
  --patch-size 128 \  
  --samples-per-epoch 2000 \  
  --batch-size 16 \  
  --epochs 50 \  
  --lr 1e-4 \  
  --num-blocks 8 \  
  --num-features 64 \  
  --res-scale 0.1
```

训练日志示例：

```
Using device: cuda  
EDSR Epoch 1/50: 100% |██████████| 125/125 [00:35<00:00, loss=0.038512]  
EDSR Epoch 001 | loss=0.046721 | Set5=28.70 dB | Set14=26.95 dB  
Saved best EDSR model: mean PSNR = 27.83 dB
```

EDSR 比 SRCNN 更深，且内部包含 PixelShuffle 上采样，因此训练会更慢，显存占用也更大。如果训练时显存不足，优先减小：

```
--batch-size  
--num-blocks  
--num-features  
--patch-size
```

训练完成后执行测试：

```
python src/test_edsr.py \  
  --checkpoint outputs/checkpoints_edsr/best.pth \  
  --set5-dir data/benchmark/Set5/HR \  
  --set14-dir data/benchmark/Set14/HR \  
  --output-dir outputs/results_edsr \  
  --scale 4
```

测试输出示例：

```
Set5 Bicubic PSNR: 28.10 dB  
Set5 EDSR      PSNR: 29.05 dB  
Set5 Improvement: 0.95 dB  
  
Set14 Bicubic PSNR: 26.40 dB  
Set14 EDSR     PSNR: 27.20 dB  
Set14 Improvement: 0.80 dB
```

注意：上面的数值只是示例，不是保证结果。实际效果和训练轮数、模型大小、数据处理方式、随机初始化等因素有关。

测试完成后查看：

```
outputs/results_edsr/Set5/  
outputs/results_edsr/Set14/
```

对比图内容为：

```
Bicubic | EDSR | HR
```

## 10. 实验结果记录

本节给出实验报告中需要记录和分析的内容。建议同学们不仅记录最终数值，还要结合图像细节进行观察，因为超分辨率任务不能只看一个 PSNR 数字。

### 10.1 记录 Set5 和 Set14 上的 PSNR 结果（按手册给出的训练参数配置）

首先记录 Bicubic、SRCNN 和 EDSR 在两个测试数据集上的平均 PSNR。

| 数据集   | Bicubic PSNR / dB | SRCNN PSNR / dB | EDSR PSNR / dB |
|-------|-------------------|-----------------|----------------|
| Set5  | XXX               | XXX             | XXX            |
| Set14 | XXX               | XXX             | XXX            |

表格中的数值只是示例，同学们需要填写自己实际运行得到的结果。

记录表格后，需要回答以下问题：

- 1. SRCNN 相比 Bicubic 是否有提升？
- 2. EDSR 相比 SRCNN 是否有进一步提升？
- 3. Set5 和 Set14 上的提升是否一致？
- 4. 如果某个模型 PSNR 没有明显提升，可能是什么原因？

### 10.2 记录测试集中提升最明显的图像

在 Set5 和 Set14 两个测试数据集中，分别选择 3 张你认为超分辨率提升效果最明显的图像进行记录。选择时可以参考 PSNR 提升，也可以参考肉眼观察到的边缘、纹理和细节改善。

每张图建议展示 5 个版本：

```
低分辨率 LR | Bicubic 放大图 | SRCNN 输出 | EDSR 输出 | HR 原图
```

建议记录格式如下：

| 数据集   | 图像名称  | Bicubic PSNR | SRCNN PSNR | EDSR PSNR | 主要观察 |
|-------|-------|--------------|------------|-----------|------|
| Set5  | 示例图 1 | XXX          | XXX        | XXX       | XXX  |
| Set5  | 示例图 2 | ...          | ...        | ...       | ...  |
| Set5  | 示例图 3 | ...          | ...        | ...       | ...  |
| Set14 | 示例图 1 | ...          | ...        | ...       | ...  |
| Set14 | 示例图 2 | ...          | ...        | ...       | ...  |
| Set14 | 示例图 3 | ...          | ...        | ...       | ...  |

注意：这里要求的是两个测试数据集各选 3 张，因此总共应记录 6 张代表性图像。

分析时可以重点观察：

- 1. 文字、线条、轮廓是否更清楚。
- 2. 纹理区域是否比 Bicubic 更自然。
- 3. SRCNN 和 EDSR 哪个结果更锐利。
- 4. 是否出现过度锐化、颜色偏移或伪影。
- 5. PSNR 较高的图像是否一定看起来最好。

实现提示：

- 前面的 `test.py` 和 `test_edsr.py` 已经会保存模型输出对比图。
- `dataset_edsr.py` 已经会返回 EDSR 使用的小尺寸 LR；如果想把 LR 小图也放进最终对比图，可以在 `test_edsr.py` 中把 `batch["lr"]` 转成图片后保存。
- SRCNN 使用的是 Bicubic 放大后的输入，EDSR 使用的是小尺寸 LR 输入；整理 5 张图时要注意两者输入形式不同。
- 如果想把 5 张图拼成一张完整对比图，可以参考 `utils.py` 中 `save_compare_image` 的写法，把画布宽度从 `width * 3` 改成 `width * 5`。
- 这部分鼓励同学们自己修改保存结果的代码，不直接给出完整改法。

### 10.3 更改训练超参数并比较结果

在完成默认参数训练后，同学们需要尝试修改训练超参数，观察不同参数对模型效果、训练速度和显存占用的影响。

可以自由调整的参数包括：

| 参数                               | 含义                     | 可以尝试的取值              |
|----------------------------------|------------------------|----------------------|
| <code>--batch-size</code>        | 每次训练使用多少个图像块           | 4、8、16、32等           |
| <code>--patch-size</code>        | 从 HR 图像中裁剪出的训练图像块大小    | 96、128、256、512等      |
| <code>--lr</code>                | 学习率，控制每次参数更新幅度         | 1e-3、5e-4、1e-4、5e-5等 |
| <code>--epochs</code>            | 训练轮数                   | 5、10、20、50等          |
| <code>--samples-per-epoch</code> | 每个 epoch 随机采样多少个 patch | 500、1000、2000、5000等  |
| <code>--num-blocks</code>        | EDSR 残差块数量             | 8、16、32等             |
| <code>--num-features</code>      | EDSR 特征通道数             | 64、128、256等          |

建议不要一次修改太多参数。更推荐采用“控制变量”的方式：每次只改变一个参数，其余参数保持不变，这样更容易判断到底是哪一个参数造成了结果变化，也可以根据自己的设备条件自由设计更多组合（不需要测试完所有的组合，选择下面的部分问题进行实验验证即可）。

分析时可以重点思考：

- 1. `batch-size` 变大后，训练是否更稳定？显存占用是否明显增加？
- 2. `patch-size` 变大后，模型是否能看到更多上下文信息？训练速度是否变慢？
- 3. `lr` 太大时，loss 是否震荡甚至不下降？
- 4. `lr` 太小时，训练是否很稳定但收敛很慢？
- 5. `epochs` 增加后，PSNR 是否持续提升，还是后期提升变小？
- 6. `samples-per-epoch` 增加后，每个 epoch 是否更耗时？最终结果是否更好？
- 7. 对 EDSR 来说，`num-blocks` 和 `num-features` 增大后，效果是否一定提升？

## 10.4 使用自己实拍图片进行测试

除标准测试集外，请选择 3 张自己拍摄的图片进行额外测试。可以使用手机拍摄校园、书本、建筑、植物、桌面物体等场景。

由于自己拍摄的图片通常没有现成的低分辨率版本，可以按下面方式构造测试输入：

- 1. 先把实拍原图作为“参考高质量图像”。
- 2. 将原图下采样，例如宽和高都缩小为原来的 `1/4`。
- 3. 从同一张低质量图像出发，分别按 SRCNN 和 EDSR 的输入要求进行推理。
- 4. 观察模型输出与原图之间的差异。

报告中建议记录：

| 图像编号  | 场景内容   | SRCNN 观察 | EDSR 观察 | 哪个结果更好 | 可能原因 |
|-------|--------|----------|---------|--------|------|
| 自拍图 1 | 例如建筑边缘 | ...      | ...     | ...    | ...  |
| 自拍图 2 | 例如书本文字 | ...      | ...     | ...    | ...  |
| 自拍图 3 | 例如植物纹理 | ...      | ...     | ...    | ...  |

实现提示：

- 可以参考 `infer.py` 和 `test_edsr.py`，把自己的低分辨率图片输入模型。
- 如果希望公平比较 SRCNN 和 EDSR，需要从同一张低分辨率图片出发。
- SRCNN 分支需要先把低分辨率图像 Bicubic 放大到目标尺寸，再输入 SRCNN。
- EDSR 分支不需要提前 Bicubic 放大，应直接把小尺寸低分辨率图像输入 EDSR，由网络内部 PixelShuffle 上采样。
- 如果想从高质量原图自动生成低分辨率图，可以用 Pillow 的 `resize` 先缩小图像。
- 实拍图片可能和 DIV2K、Set5、Set14 的分布不同，因此模型效果可能不如标准测试集稳定。
- 观察时不要只关注“是否更锐利”，还要注意是否出现不自然纹理、边缘振铃或颜色变化。

伪代码框架如下：

```
准备：
    选择 3 张自己拍摄的高清图片
    分别记为 image_1, image_2, image_3

对每一张实拍图片：
    1. 读取原始高清图像
        # 这张图可以看作参考图像，用来观察模型结果是否接近真实细节

    2. 将原图下采样到较小尺寸
        # 例如 scale=4 时，宽和高都缩小为原来的 1/4
        # 这一步用于模拟低分辨率输入

    3. 保存下采样后的低分辨率图像
        # 后面的 SRCNN 分支和 EDSR 分支都应该从这张低分辨率图出发
        # 这样比较才公平

    4. 使用 Bicubic 将低分辨率图像放大回原图尺寸
        # 这是传统插值结果，可以作为基线方法
        # 也是 SRCNN 分支的输入

    5. 将 Bicubic 放大后的图像输入 SRCNN 推理脚本
        # 得到 SRCNN 的超分辨率结果
        # SRCNN 本身不负责改变图像尺寸

    6. 将同一张低分辨率图像输入 EDSR 推理脚本
        # 得到 EDSR 的超分辨率结果
        # PixelShuffle 版 EDSR 会在网络内部完成上采样
```

7. 保存并整理对比图
- # 建议排列顺序：
- # 低分辨率图 | Bicubic 放大图 | SRCNN 结果 | EDSR 结果 | 原始高清图
8. 观察和记录结果
- # 重点分析边缘、文字、纹理、颜色和伪影
- # 比较 SRCNN 和 EDSR 哪个更适合这张真实图片

这部分的重点是理解模型泛化能力：模型在标准测试集上表现好，不代表它在所有真实图像上都一定表现最好。

## 10.5 进阶任务：尝试更新的超分辨率网络（选做）

如果同学们之前已经学习过更多深度学习内容，或者对 PyTorch 项目结构比较熟悉，可以额外选择一个更新的超分辨率网络进行训练或测试，并与 Bicubic、SRCNN、EDSR 进行比较。

可以考虑的模型方向包括：

| 模型          | 特点                           |
|-------------|------------------------------|
| RCAN        | 使用通道注意力机制，能增强重要特征            |
| RDN         | 使用密集连接，强调多层特征复用              |
| ESRGAN      | 更关注视觉真实感，常用于生成更锐利的图像         |
| SwinIR      | 使用 Transformer 结构，是较新的图像复原模型 |
| Real-ESRGAN | 更适合真实退化图像，常用于实际图片增强          |

报告中建议说明：

1. 选择了哪个模型。
2. 为什么选择这个模型。
3. 是否使用预训练权重。
4. 是否重新训练，训练数据和参数是什么。
5. 在 Set5、Set14 或自拍图片上的表现如何。
6. 它相比 SRCNN 和 EDSR 的优势与不足是什么。

注意：这部分是进阶任务，不要求所有同学完成。完成时可以使用开源实现或预训练模型，但需要在报告中清楚说明来源、运行方式和比较方法。

## 11. 常见问题

### 11.1 训练很慢怎么办

可以减少训练量：

```
--samples-per-epoch 500
--epochs 10
--batch-size 8
```

如果有 GPU，建议不要加 `--cpu`。

## 11.2 显存不足怎么办

优先减小 batch size：

```
--batch-size 4
```

如果还不够，再减小 patch size：

```
--patch-size 64
```

## 11.3 PSNR 没有提升怎么办

可以检查：

1. 数据集是否放在正确目录。
2. Set5 和 Set14 的 HR 图像是否复制正确。
3. 训练轮数是否太少。
4. 学习率是否过大或过小。
5. 输出图片是否正常保存。

## 11.4 为什么不直接输入小尺寸 LR

本实验采用经典 SRCNN 的简化思路：先 Bicubic 放大，再用 CNN 修复。

这样做的好处是：

- 输入和输出尺寸相同，代码更简单。
- 不需要在网络中写上采样层。
- 更适合初学者理解图像到图像的映射。

## 11.5 为什么训练用 patch

DIV2K 图像很大，直接用整张图训练会占用大量显存。

随机裁剪 patch 的好处：

- 显存占用小。
- 训练速度更快。
- 同一张图可以裁剪出很多不同训练样本。